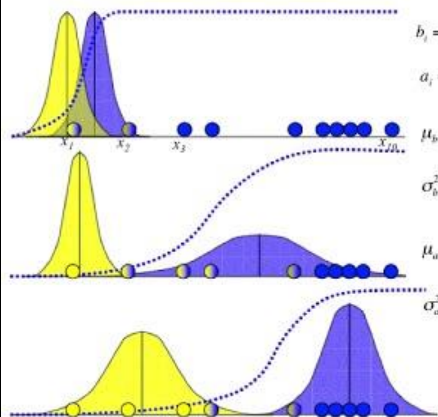


EXPECTATION MAXIMIZATION

EM: 1-d example



$$P(x_i | b) = \frac{1}{\sqrt{2\pi\sigma_b^2}} \exp\left[-\frac{(x_i - \mu_b)^2}{2\sigma_b^2}\right]$$

$$b_i = P(b | x_i) = \frac{P(x_i | b)P(b)}{P(x_i | b)P(b) + P(x_i | a)P(a)}$$

$$a_i = P(a | x_i) = 1 - b_i$$

$$\mu_b = \frac{b_1 x_1 + b_2 x_2 + \dots + b_n x_n}{b_1 + b_2 + \dots + b_n}$$

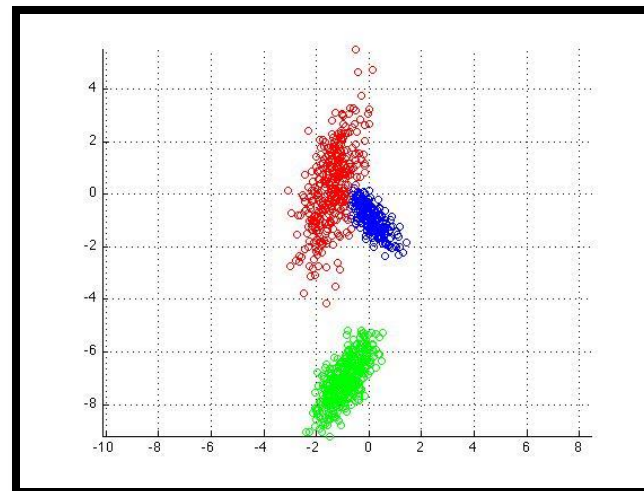
$$\sigma_b^2 = \frac{b_1 (x_1 - \mu_b)^2 + \dots + b_n (x_n - \mu_b)^2}{b_1 + b_2 + \dots + b_n}$$

$$\mu_a = \frac{a_1 x_1 + a_2 x_2 + \dots + a_n x_n}{a_1 + a_2 + \dots + a_n}$$

$$\sigma_a^2 = \frac{a_1 (x_1 - \mu_a)^2 + \dots + a_n (x_n - \mu_a)^2}{a_1 + a_2 + \dots + a_n}$$

could also estimate priors:
 $P(b) = (b_1 + b_2 + \dots + b_n) / n$
 $P(a) = 1 - P(b)$

Copyright © 2004 Victor L. Linsley



INTUITION

- Assume that we have two coins, C1 and C2
- Assume the bias of C1 is θ_1
(i.e., probability of getting heads with C1)
- Assume the bias of C2 is θ_2
(i.e., probability of getting heads with C2)
- We want to find θ_1, θ_2 by performing a number of trials
(i.e., coin tosses)

INTUITION

First experiment

- We choose 5 times one of the coins.
- We toss the chosen coin 10 times

	H T T T H H T H T H
	H H H H T H H H H H
	H T H H H H H T H H
	H T H T T T H H T T
	T H H H T H H H T H

$$\theta_1 = \frac{\text{number of heads using C1}}{\text{total number of flips using C1}}$$

$$\theta_2 = \frac{\text{number of heads using C2}}{\text{total number of flips using C2}}$$

INTUITION



H T T T H H T H T H



H H H H T H H H H H



H T H H H H H T H H



H T H T T T H H T T



T H H H T H H H T H

Coin A	Coin B
	5 H, 5 T
9 H, 1 T	
8 H, 2 T	
	4 H, 6 T
7 H, 3 T	
24 H, 6 T	9 H, 11 T

$$\theta_1 = \frac{24}{24 + 6} = 0.8$$

$$\theta_2 = \frac{9}{9 + 11} = 0.45$$

INTUITION

Assume a more challenging problem

H T T T H H T H T H

H H H H T H H H H H

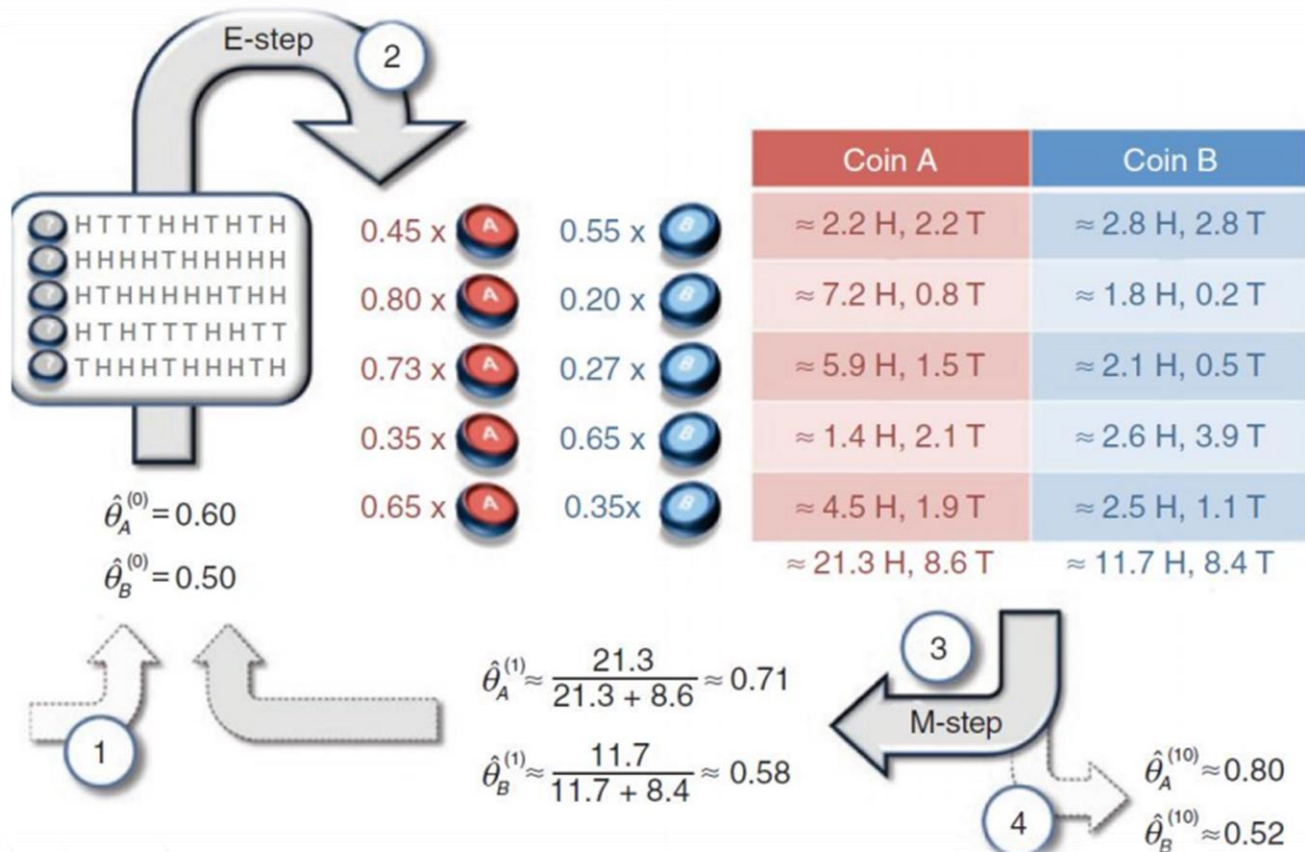
H T H H H H H T H H

H T H T T T H H T T

T H H H T H H H T H

- We do not know the identities of the coins used for each set of tosses (we treat them as hidden variables).

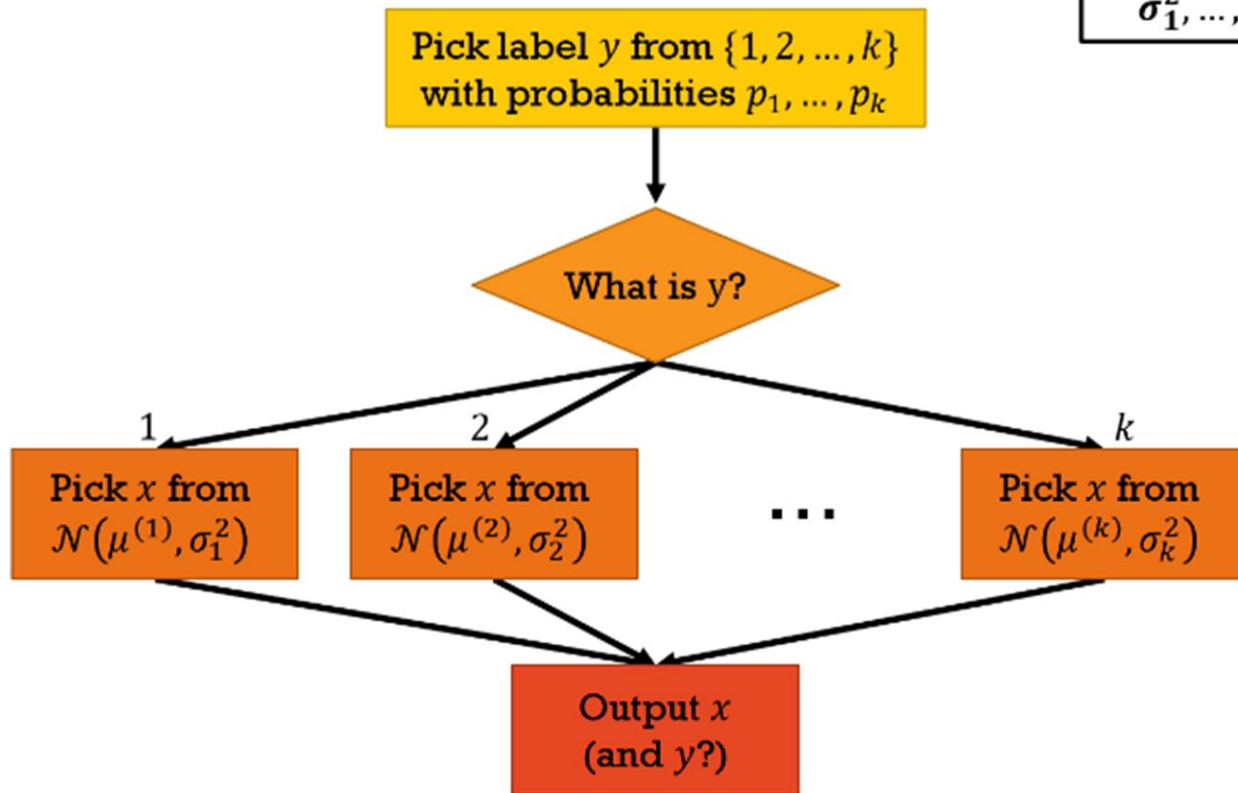
INTUITION



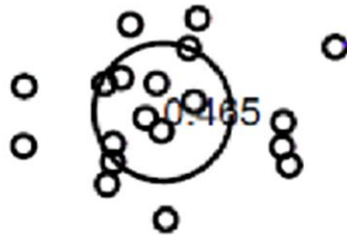
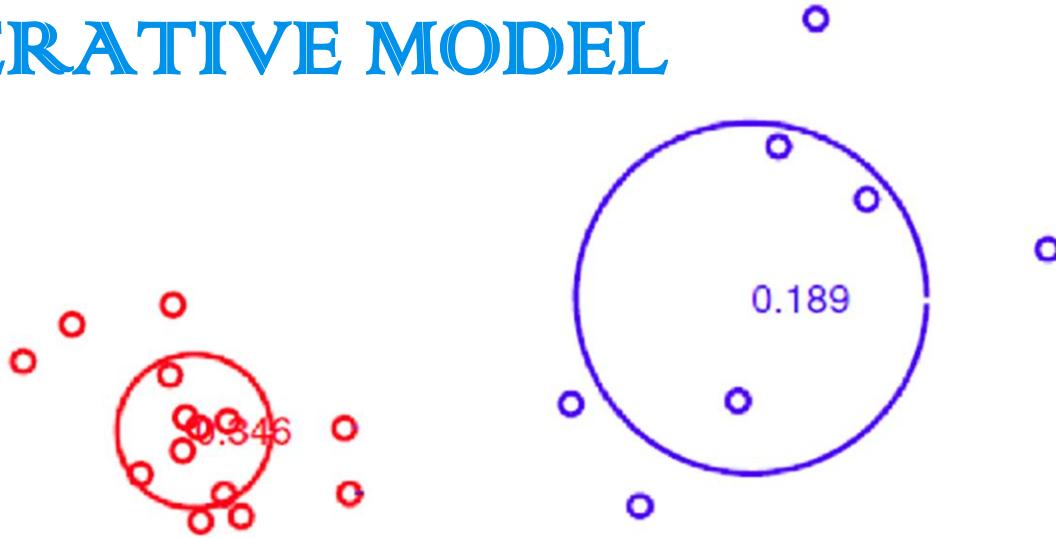
GENERATIVE MODEL

Model Parameters

$$p_1, \dots, p_k$$
$$\mu^{(1)}, \dots, \mu^{(k)}$$
$$\sigma_1^2, \dots, \sigma_k^2$$



GENERATIVE MODEL



Points x – dots

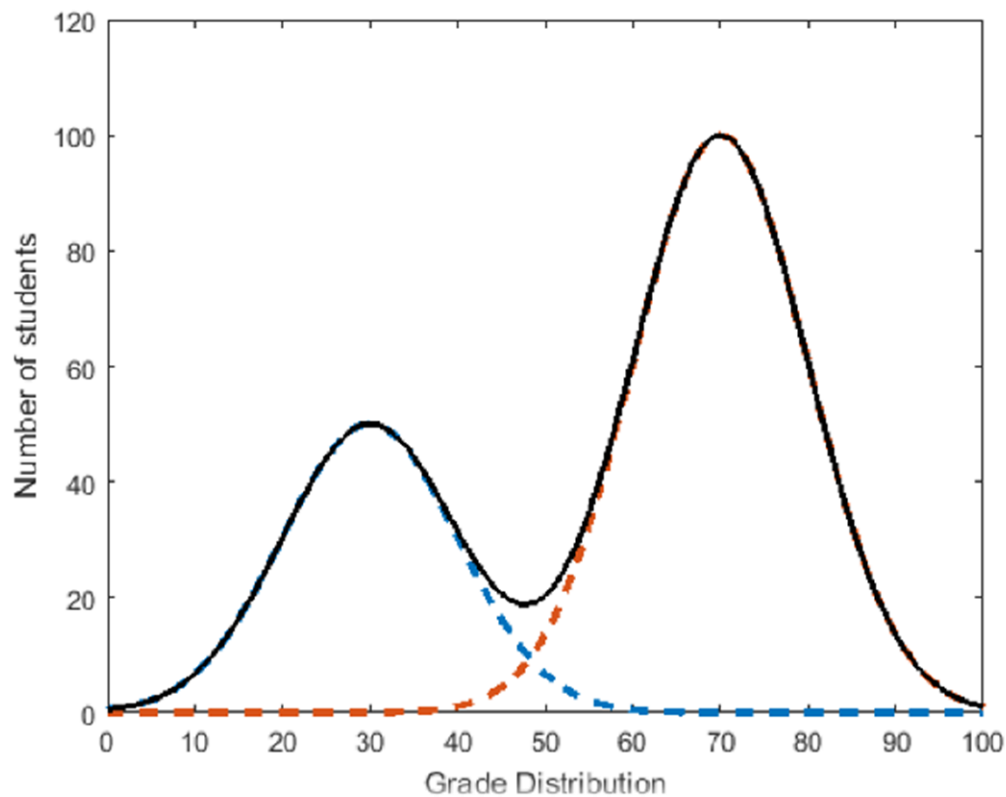
Label y – color of dots

Prior p_y – proportion of dots

Mean $\mu^{(y)}$ – center of circle

Variance σ_y^2 – size of circle

GENERATIVE MODEL



GAUSSIAN MIXTURE MODEL

Given training set $x^{(1)}, \dots, x^{(N)}$, we wish to model the data by specifying the joint distribution of X with a latent variable Z :

$$p(x, z) = p(x|z)p(z).$$

Here $Z \sim \text{Multinomial}(\pi)$, i.e.

$$P(Z = j) = \pi_j, \quad j = 1, \dots, m,$$

and $\sum_{j=1}^m \pi_j = 1$, and we have

$$X \mid \{Z = j\} \sim \mathcal{N}(\mu_j, \Sigma_j).$$

GAUSSIAN MIXTURE MODEL

- This means that the log-likelihood of the data is given by

$$\begin{aligned}\ell(\pi, \mu, \Sigma) &= \sum_{i=1}^N \log p(x^{(i)}) = \sum_{i=1}^N \log \sum_{j=1}^m p(x^{(i)} | Z = j) P(Z = j) \\ &= \sum_{i=1}^N \log \sum_{j=1}^m \pi_j \left(C_j \exp^{-\frac{1}{2} \langle x^{(i)} - \mu_j, \Sigma_j^{-1} (x^{(i)} - \mu_j) \rangle} \right)\end{aligned}$$

Normalization constant! ←

- Unfortunately, there is no closed-form solution to optimizing this log-likelihood. Let's see why by examining the conditions we get when we try to optimize $\ell(\pi, \mu, \Sigma)$.

GAUSSIAN MIXTURE MODEL

- Differentiating with respect to μ_k and setting to zero, we get

$$\sum_{i=1}^N \frac{\pi_k \left(C_j \exp^{-\frac{1}{2} \langle x^{(i)} - \mu_j, \Sigma_j^{-1} (x^{(i)} - \mu_j) \rangle} \right) \Sigma^{-1} (x^{(i)} - \mu_k)}{\sum_{j=1}^m \pi_j \left(C_j \exp^{-\frac{1}{2} \langle x^{(i)} - \mu_j, \Sigma_j^{-1} (x^{(i)} - \mu_j) \rangle} \right)} = 0$$

- We will denote

$$\gamma \left(z_k^{(i)} \right) := \frac{\pi_k \left(C_j \exp^{-\frac{1}{2} \langle x^{(i)} - \mu_k, \Sigma_k^{-1} (x^{(i)} - \mu_k) \rangle} \right)}{\sum_{j=1}^m \pi_j \left(C_j \exp^{-\frac{1}{2} \langle x^{(i)} - \mu_j, \Sigma_j^{-1} (x^{(i)} - \mu_j) \rangle} \right)},$$

C_j should be C_k
in the numerator.

and rearranging, we get

$$\mu_k = \frac{\sum_{i=1}^N x^{(i)} \gamma \left(z_k^{(i)} \right)}{\sum_{i=1}^N \gamma \left(z_k^{(i)} \right)}. \quad (1)$$

GAUSSIAN MIXTURE MODEL

- Similarly, differentiating with respect to Σ_k^{-1} and setting to zero, we get

$$\Sigma_k = \frac{\sum_{i=1}^N \gamma(z_k^{(i)}) (x^{(i)} - \mu_k) (x^{(i)} - \mu_k)^T}{\sum_{i=1}^N \gamma(z_k^{(i)})}. \quad (2)$$

LAGRANGE MULTIPLIERS

- Since π satisfies the constraint $\sum_{j=1}^m \pi_j = 1$, we'll have to use Lagrange multipliers to optimize with respect to π . We have

$$\frac{d}{d\pi_k} \left[\sum_{i=1}^N \log \sum_{j=1}^m \pi_j \left(C_j \exp^{-\frac{1}{2} \langle x^{(i)} - \mu_j, \Sigma_j^{-1} (x^{(i)} - \mu_j) \rangle} \right) - \lambda \left(\sum_{j=1}^m \pi_j - 1 \right) \right] = 0,$$

- which implies that

$$\sum_{i=1}^N \frac{\left(C_k \exp^{-\frac{1}{2} \langle x^{(i)} - \mu_k, \Sigma_k^{-1} (x^{(i)} - \mu_k) \rangle} \right)}{\sum_{j=1}^m \pi_j \left(C_j \exp^{-\frac{1}{2} \langle x^{(i)} - \mu_j, \Sigma_j^{-1} (x^{(i)} - \mu_j) \rangle} \right)} = \lambda.$$

LAGRANGE MULTIPLIERS

- Multiplying by π_k on both sides, we get

$$\sum_{i=1}^N \gamma \left(z_k^{(i)} \right) = \lambda \pi_k.$$

- Summing over k gives us $\lambda = N$, and thus

$$\pi_k = \frac{1}{N} \sum_{i=1}^N \gamma \left(z_k^{(i)} \right). \quad (3)$$

GAMMA MEANING

As suggested in the notation, $\gamma(z_k^{(i)})$ represents an important quantity related to the latent variable Z . Indeed, if we compute

$$P(Z = k | x^{(i)}) = \frac{p(x^{(i)} | Z = k) P(Z = k)}{p(x^{(i)})}$$

using Bayes' rule, we get

$$\begin{aligned} P(Z = k | x^{(i)}) &= \frac{\pi_k \left(C_j \exp^{-\frac{1}{2} \langle x^{(i)} - \mu_k, \Sigma_k^{-1} (x^{(i)} - \mu_k) \rangle} \right)}{\sum_{j=1}^m \pi_j \left(C_j \exp^{-\frac{1}{2} \langle x^{(i)} - \mu_j, \Sigma_j^{-1} (x^{(i)} - \mu_j) \rangle} \right)} \\ &= \gamma(z_k^{(i)}). \end{aligned}$$

ALGORITHM

As $\gamma(z_k^{(i)})$ contains the parameters μ , Σ and π in a complex way, (1), (2) and (3) cannot be solved in closed-form. However, it suggests the following two-step algorithm:

- E-step: Set

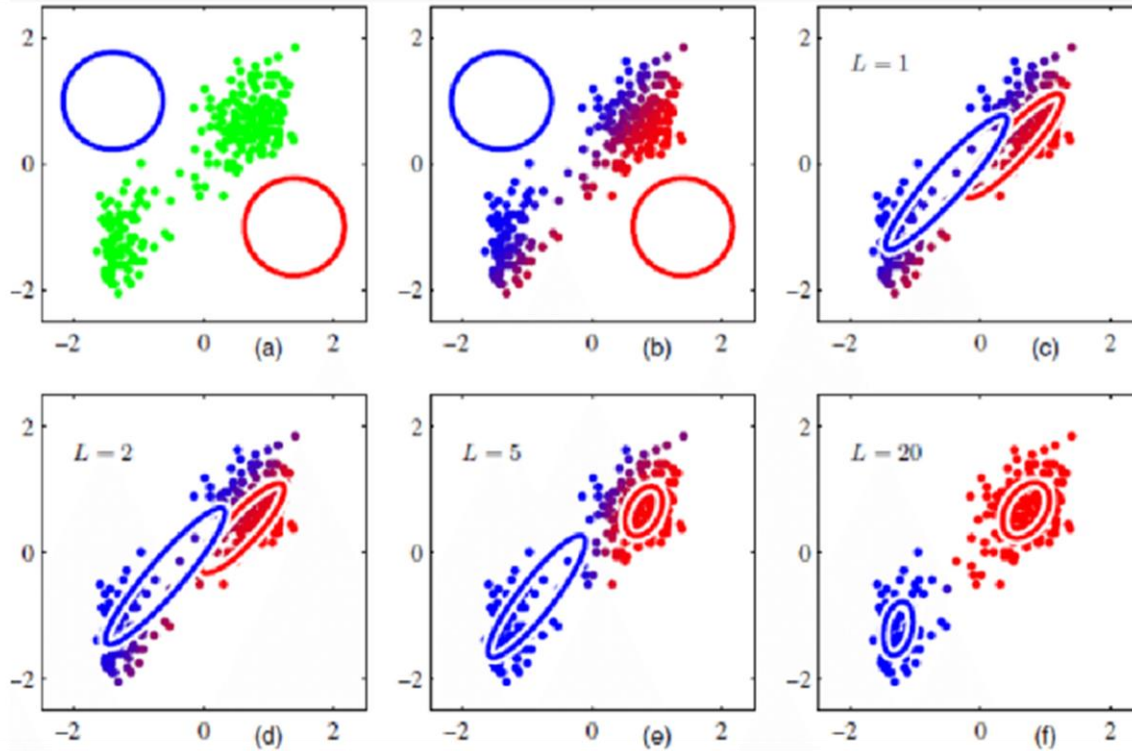
$$\gamma_{t+1}(z_k^{(i)}) = P_{\mu(t), \Sigma(t), \pi(t)}(Z = k | x^{(i)})$$

- M-step: Set

$$\begin{aligned}\pi_k(t+1) &= \frac{1}{N} \sum_{i=1}^N \gamma_{t+1}(z_k^{(i)}) \\ \mu_k(t+1) &= \frac{\sum_{i=1}^N x^{(i)} \gamma_{t+1}(z_k^{(i)})}{\sum_{i=1}^N \gamma_{t+1}(z_k^{(i)})} \\ \Sigma_k(t+1) &= \frac{\sum_{i=1}^N \gamma_{t+1}(z_k^{(i)}) (x^{(i)} - \mu_k(t+1)) (x^{(i)} - \mu_k(t+1))^T}{\sum_{i=1}^N \gamma_{t+1}(z_k^{(i)})}.\end{aligned}$$

- Repeat until convergence.

OLD FAITHFUL DATA SET

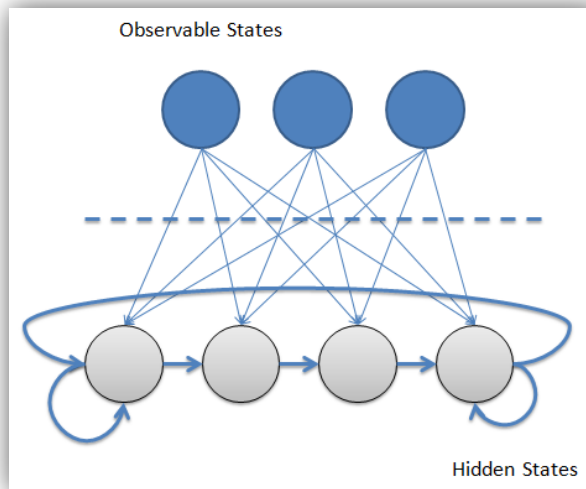
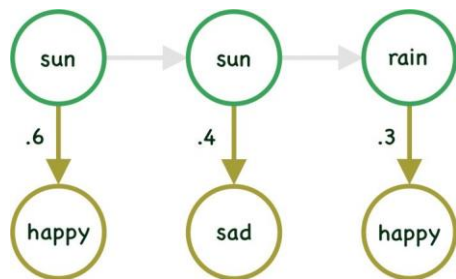


COMPARISON WITH K MEANS

- Like k-means, EM clustering may get stuck in local minima.
- Unlike k-means, the local minima are more favorable because soft labels allow points to move between clusters slowly.

EM allows for overlaps too!

HIDDEN MARKOV MODELS

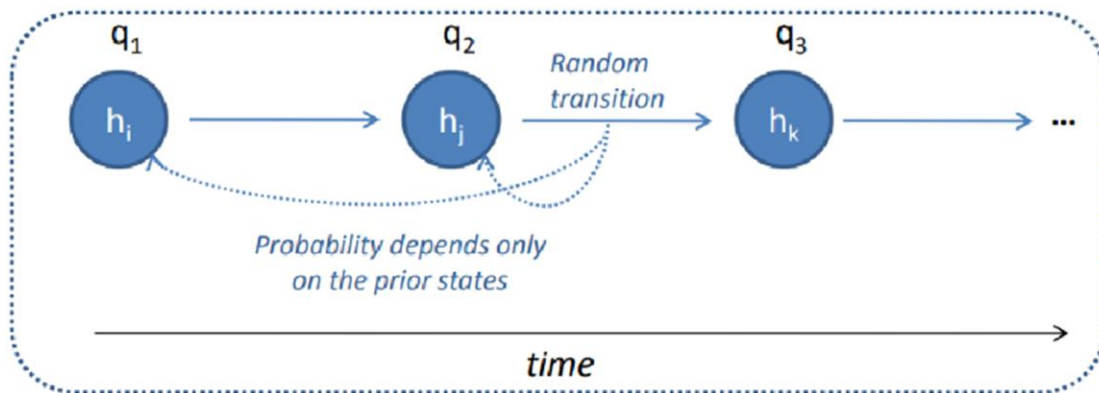


APPLICATIONS

- Speech generation, speech recognition
- Financial forecasting: stock prices, currency exchange rates etc.
- DNA sequencing
- Weather prediction
- Video analytics
- Machine translation

MARKOV CHAINS

- Markov Chains model sequential processes.
- Consider a discrete random variable q with states $\{h_1, \dots, h_n\}$.
- State of q changes randomly in discrete time steps.
- Transition probability depends only on the k previous states.
 - Markov Property



Markov Chain

MARKOV CHAINS

- Most simplest Markov chain:

- Transition Probability depends only on the previous state (i.e. $k=1$):

$$P(q_t = h_i | q_{t-1}, \dots, q_1) = P(q_t = h_i | q_{t-1})$$

- Transition Probability is time invariant:

$$P(q_t = h_i | q_{t-1}) = P(q_{t-1} = h_i | q_{t-2})$$

- In this case, the Markov chain is defined by:

- An $(n \times n)$ Matrix T containing state change probabilities:

$$T_{ij} = P(q_t = h_i | q_{t-1} = h_j)$$

- An n -dimensional vector π containing initial state probabilities:

$$\pi_i = P(q_1 = h_i)$$

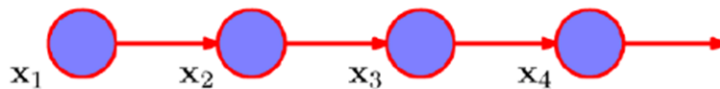
- Since π and K contain probabilities, they have to be normalized:

$$\sum_{i=1}^n \pi_i = 1 \quad \forall a : \sum_{i=1}^n K_{ai} = 1$$

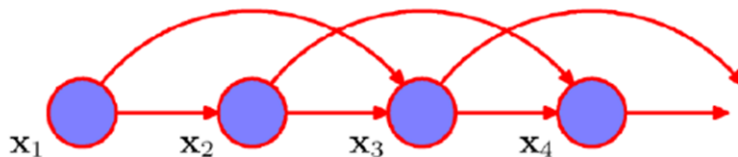
MARKOV CHAINS

- Markov Assumption

1st order
$$p(\mathbf{X}) = \prod_{i=1}^n p(X_i | X_{i-1})$$



2nd order
$$p(\mathbf{X}) = \prod_{i=1}^n p(X_i | X_{i-1}, X_{i-2})$$



PRELUDE

- Given observations $(O_1, \dots, O_n)$, hidden states $(S_1, \dots, S_n)$, we want to define a model by providing structure to our joint distribution $p(O_1, \dots, O_n, S_1, \dots, S_n)$. We have

$$p(O_1, \dots, O_n, S_1, \dots, S_n) = p(S_1, \dots, S_n)p(O_1, \dots, O_n | S_1, \dots, S_n)$$

using chain rule.

- Repeated application of chain rule to $p(S_1, \dots, S_n)$ yields

$$\begin{aligned} p(S_1, \dots, S_n) &= p(S_1, \dots, S_{n-1})p(S_n | S_1, \dots, S_{n-1}) \\ &= p(S_1, \dots, S_{n-2})p(S_{n-1} | S_1, \dots, S_{n-2})p(S_n | S_1, \dots, S_{n-1}) \\ &\quad \vdots \\ &= p(S_1)p(S_2 | S_1)p(S_3 | S_1, S_2) \cdots p(S_n | S_1, \dots, S_{n-1}). \end{aligned}$$

PRELUDE

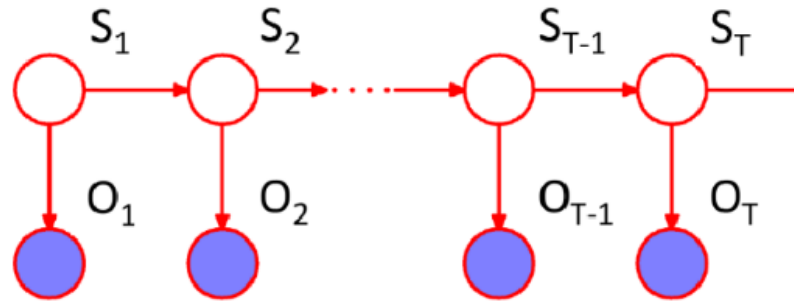
- Similarly for $p(O_1, \dots, O_n | S_1, \dots, S_n)$, we have

$$\begin{aligned} p(O_1, \dots, O_n | S_1, \dots, S_n) \\ = p(O_1 | S_1, \dots, S_n) p(O_2 | O_1, S_1, \dots, S_n) \cdots p(O_n | O_1, \dots, O_{n-1}, S_1, \dots, S_n). \end{aligned}$$

- So far we have enforced no assumptions; we will do so in the next few slides.

HIDDEN MARKOV MODEL

- Distributions that characterize sequential data with few parameters but are not limited by strong Markov assumptions.



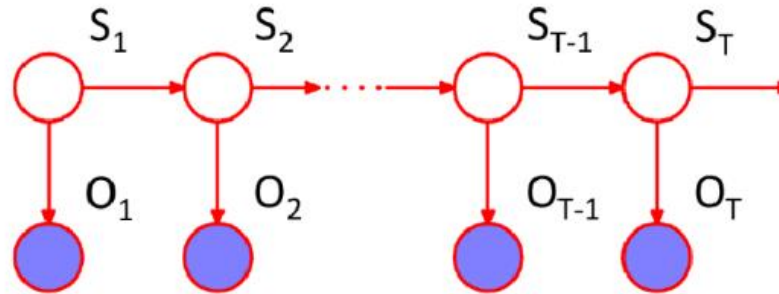
Observation space

$$O_t \in \{y_1, y_2, \dots, y_K\}$$

Hidden states

$$S_t \in \{1, \dots, I\}$$

HIDDEN MARKOV MODEL



$$p(S_1, \dots, S_T, O_1, \dots, O_T) = \prod_{t=1}^T p(O_t | S_t) \prod_{t=1}^T p(S_t | S_{t-1})$$

1st order Markov assumption on hidden states $\{S_t\}$ $t = 1, \dots, T$
(can be extended to higher order).

Note: O_t depends on all previous observations $\{O_{t-1}, \dots, O_1\}$

MODEL PARAMETERS

- Parameters – stationary/homogeneous markov model (independent of time t)

Initial probabilities

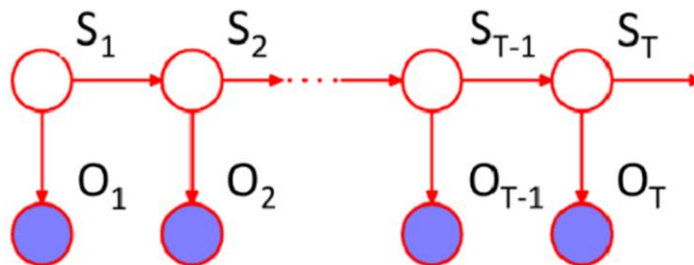
$$p(S_1 = i) = \pi_i$$

Transition probabilities

$$p(S_t = j | S_{t-1} = i) = p_{ij}$$

Emission probabilities

$$p(O_t = y | S_t = i) = q_i^y$$



$$p(\{S_t\}_{t=1}^T, \{O_t\}_{t=1}^T) =$$

$$p(S_1) \prod_{t=2}^T p(S_t | S_{t-1}) \prod_{t=1}^T p(O_t | S_t)$$

EXAMPLE

- The Dishonest Casino

A casino has two die:

Fair die

$$P(1) = P(2) = P(3) = P(5) = P(6) = 1/6$$

Loaded die

$$P(1) = P(2) = P(3) = P(5) = 1/10$$

$$P(6) = \frac{1}{2}$$

Casino player switches back-&-forth between fair and loaded die once every 20 turns



EXAMPLE

GIVEN: A sequence of rolls by the casino player

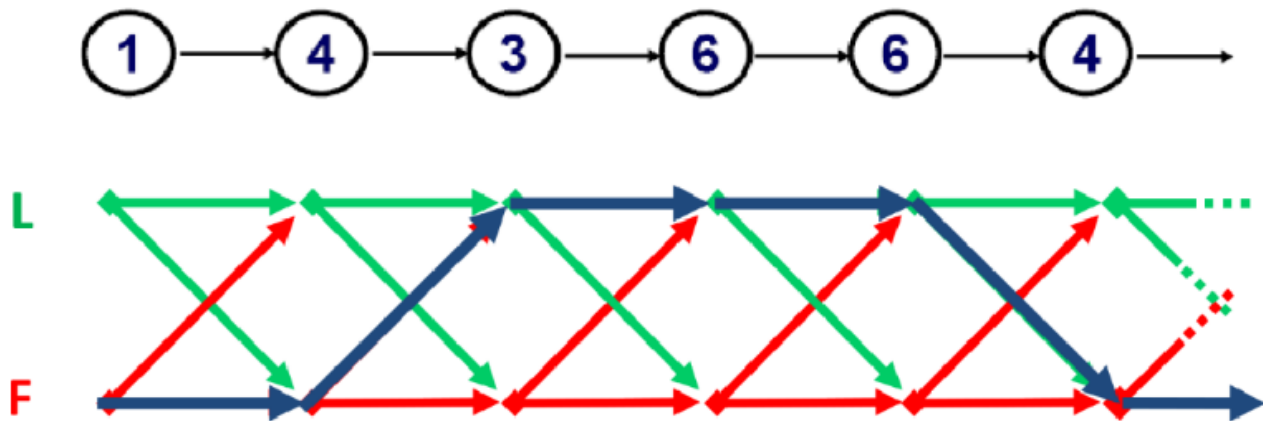
1245526462146146136136661664661636616366163616515615115146123562344

QUESTION

- How likely is this sequence, given our model of how the casino works?
 - This is the **EVALUATION** problem in HMMs
- What portion of the sequence was generated with the fair die, and what portion with the loaded die?
 - This is the **DECODING** question in HMMs
- How “loaded” is the loaded die? How “fair” is the fair die? How often does the casino player change from fair to loaded, and back?
 - This is the **LEARNING** question in HMMs

EXAMPLE

- Observed sequence: $\{O_t\}_{t=1}^T$

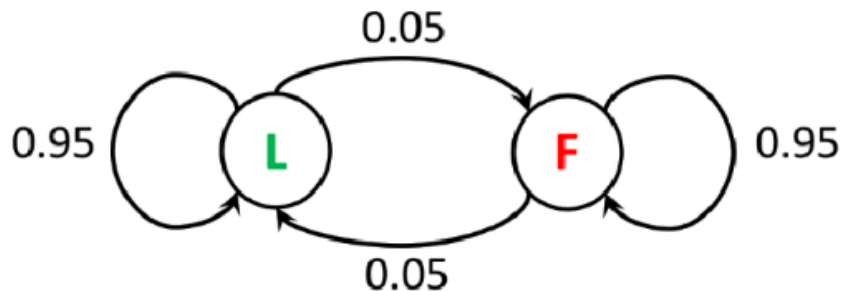


- Hidden sequence $\{S_t\}_{t=1}^T$ (or segmentation):



EXAMPLE

- Switch between **F** and **L** once every 20 turns ($1/20 = 0.05$)



- HMM Parameters

Initial probs

$$P(S_1 = \text{L}) = 0.5 = P(S_1 = \text{F})$$

Transition probs

$$P(S_t = \text{L/F} | S_{t-1} = \text{L/F}) = 0.95$$

$$P(S_t = \text{F/L} | S_{t-1} = \text{L/F}) = 0.05$$

Emission probabilities

$$P(O_t = y | S_t = \text{F}) = 1/6 \quad y = 1, 2, 3, 4, 5, 6$$

$$P(O_t = y | S_t = \text{L}) = 1/10 \quad y = 1, 2, 3, 4, 5$$
$$= 1/2 \quad y = 6$$

THREE MAIN LEARNING PROBLEMS

- **Evaluation** – Given HMM parameters & observation seqn $\{O_t\}_{t=1}^T$
find $p(\{O_t\}_{t=1}^T)$ prob of observed sequence
- **Decoding** – Given HMM parameters & observation seqn $\{O_t\}_{t=1}^T$
find $\arg \max_{s_1, \dots, s_T} p(\{S_t\}_{t=1}^T | \{O_t\}_{t=1}^T)$ most probable
sequence of hidden states
- **Learning** – Given HMM with unknown parameters and $\{O_t\}_{t=1}^T$
observation sequence
find $\arg \max_{\theta} p(\{O_t\}_{t=1}^T | \theta)$ parameters that maximize
likelihood of observed data

THREE MAIN LEARNING PROBLEMS

- **Evaluation** – What is the probability of the observed sequence? **Forward Algorithm**
- **Decoding** – What is the probability that the third roll was loaded given the observed sequence? **Forward-Backward Algorithm**
 - What is the most likely die sequence given the observed sequence? **Viterbi Algorithm**
- **Learning** – Under what parameterization is the observed sequence most probable? **Baum-Welch Algorithm (EM)**

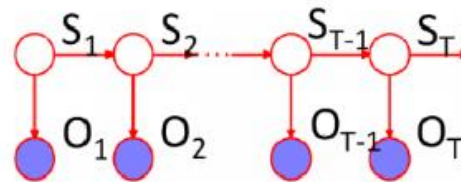
EVALUATION: FORWARD ALGO

- Given HMM parameters $p(S_1), p(S_t|S_{t-1}), p(O_t|S_t)$ & observation sequence $\{O_t\}_{t=1}^T$

find probability of observed sequence

$$p(\{O_t\}_{t=1}^T) = \sum_{S_1, \dots, S_T} p(\{O_t\}_{t=1}^T, \{S_t\}_{t=1}^T)$$

$$= \sum_{S_1, \dots, S_T} p(S_1) \prod_{t=2}^T p(S_t|S_{t-1}) \prod_{t=1}^T p(O_t|S_t)$$



requires summing over all possible hidden state values at all times – K^T exponential # terms!

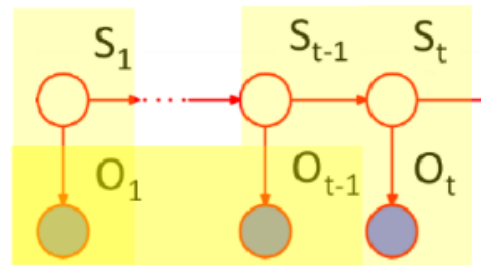
Instead:
$$p(\{O_t\}_{t=1}^T) = \sum_k \underbrace{p(\{O_t\}_{t=1}^T, S_T = k)}_{\alpha_T^k \text{ Compute recursively}}$$

EVALUATION: FORWARD ALGO

$$p(\{O_t\}_{t=1}^T) = \sum_k p(\{O_t\}_{t=1}^T, S_T = k) = \sum_k \alpha_T^k$$

Compute forward probability α_t^k recursively over t

$$\alpha_t^k := p(O_1, \dots, O_t, S_t = k)$$



Introduce S_{t-1}

Chain rule

Markov assumption

$$= p(O_t | S_t = k) \sum_i \alpha_{t-1}^i p(S_t = k | S_{t-1} = i)$$

EVALUATION: FORWARD ALGO

Can compute α_t^k for all k, t using dynamic programming:

- Initialize: $\alpha_1^k = p(O_1 | S_1 = k) p(S_1 = k)$ for all k

- Iterate: for $t = 2, \dots, T$

$$\alpha_t^k = p(O_t | S_t = k) \sum_i \alpha_{t-1}^i p(S_t = k | S_{t-1} = i) \quad \text{for all } k$$

- Termination: $p(\{O_t\}_{t=1}^T) = \sum_k \alpha_T^k$

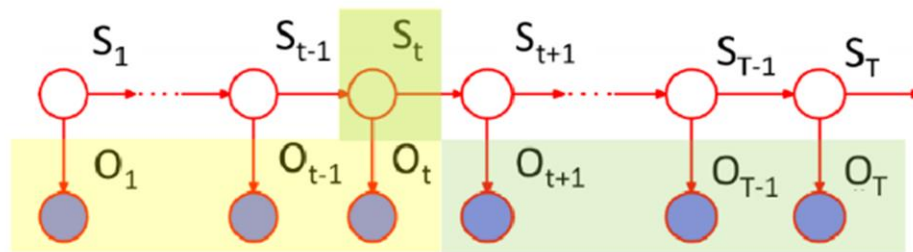
DECODING: BACKWARD ALGO

- Given HMM parameters $p(S_1), p(S_t|S_{t-1}), p(O_t|S_t)$ & observation sequence $\{O_t\}_{t=1}^T$

find probability that hidden state at time t was k $p(S_t = k | \{O_t\}_{t=1}^T)$

$$\begin{aligned} p(S_t = k, \{O_t\}_{t=1}^T) &= p(O_1, \dots, O_t, S_t = k, O_{t+1}, \dots, O_T) \\ &= \underbrace{p(O_1, \dots, O_t, S_t = k)}_{\alpha_t^k} \underbrace{p(O_{t+1}, \dots, O_T | S_t = k)}_{\beta_t^k} \end{aligned}$$

Compute recursively

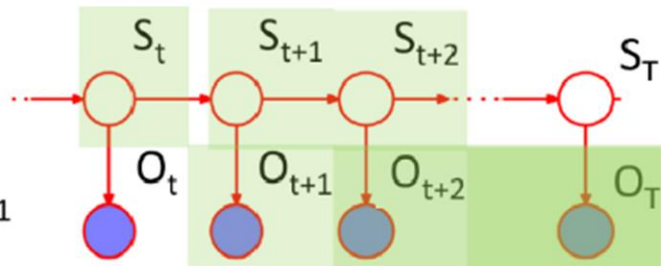


DECODING: BACKWARD ALGO

$$p(S_t = k, \{O_t\}_{t=1}^T) = p(O_1, \dots, O_t, S_t = k) p(O_{t+1}, \dots, O_T | S_t = k) = \alpha_t^k \beta_t^k$$

Compute forward probability β_t^k recursively over t

$$\beta_t^k := p(O_{t+1}, \dots, O_T | S_t = k)$$



Introduce S_{t+1}

Chain rule

Markov assumption

$$= \sum_i p(S_{t+1} = i | S_t = k) p(O_{t+1} | S_{t+1} = i) \beta_{t+1}^i$$

DECODING: BACKWARD ALGO

Can compute β_t^k for all k, t using dynamic programming:

- Initialize: $\beta_T^k = 1$ for all k

- Iterate: for $t = T-1, \dots, 1$

$$\beta_t^k = \sum_i p(S_{t+1} = i | S_t = k) p(O_{t+1} | S_{t+1} = i) \beta_{t+1}^i \quad \text{for all } k$$

- Termination: $p(S_t = k, \{O_t\}_{t=1}^T) = \alpha_t^k \beta_t^k$

$$p(S_t = k | \{O_t\}_{t=1}^T) = \frac{p(S_t = k, \{O_t\}_{t=1}^T)}{p(\{O_t\}_{t=1}^T)} = \frac{\alpha_t^k \beta_t^k}{\sum_i \alpha_t^i \beta_t^i}$$

DECODING: VITERBI ALGO

- Most likely state assignment at time t

$$\arg \max_k p(S_t = k | \{O_t\}_{t=1}^T) = \arg \max_k \alpha_t^k \beta_t^k$$

E.g. Which die was most likely used by the casino in the third roll given the observed sequence?

- Most likely assignment of state sequence

$$\arg \max_{\{S_t\}_{t=1}^T} p(\{S_t\}_{t=1}^T | \{O_t\}_{t=1}^T)$$

E.g. What was the most likely sequence of die rolls used by the casino given the observed sequence?

Not the same solution !

MLA of x?
MLA of (x,y)?

x	y	P(x,y)
0	0	0.35
0	1	0.05
1	0	0.3
1	1	0.3

DECODING: VITERBI ALGO

- Given HMM parameters $p(S_1), p(S_t|S_{t-1}), p(O_t|S_t)$ & observation sequence $\{O_t\}_{t=1}^T$

find most likely assignment of state sequence

$$\begin{aligned}\arg \max_{\{S_t\}_{t=1}^T} p(\{S_t\}_{t=1}^T | \{O_t\}_{t=1}^T) &= \arg \max_{\{S_t\}_{t=1}^T} p(\{S_t\}_{t=1}^T, \{O_t\}_{t=1}^T) \\ &= \arg \max_k \underbrace{\max_{\{S_t\}_{t=1}^{T-1}} p(S_T = k, \{S_t\}_{t=1}^{T-1}, \{O_t\}_{t=1}^T)}_{V_T^k}\end{aligned}$$

Compute recursively

V_T^k - probability of most likely sequence of states ending at state $S_T = k$

DECODING: VITERBI ALGO

$$\max_{\{S_t\}_{t=1}^T} p(\{S_t\}_{t=1}^T, \{O_t\}_{t=1}^T) = \max_k V_T^k$$

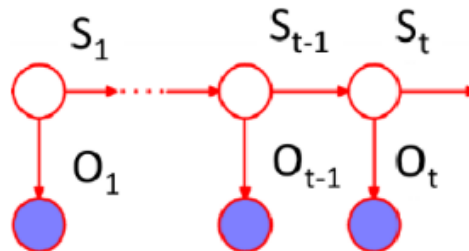
Compute probability V_t^k recursively over t

$$V_t^k := \max_{S_1, \dots, S_{t-1}} p(S_t = k, S_1, \dots, S_{t-1}, O_1, \dots, O_t)$$

·
·
·

Bayes rule

Markov assumption



$$= p(O_t | S_t = k) \max_i p(S_t = k | S_{t-1} = i) V_{t-1}^i$$

DECODING: VITERBI ALGO

Can compute V_t^k for all k, t using dynamic programming:

- Initialize: $V_1^k = p(O_1|S_1=k)p(S_1 = k)$ for all k

- Iterate: for $t = 2, \dots, T$

$$V_t^k = p(O_t|S_t = k) \max_i p(S_t = k|S_{t-1} = i) V_{t-1}^i \quad \text{for all } k$$

- Termination: $\max_{\{S_t\}_{t=1}^T} p(\{S_t\}_{t=1}^T, \{O_t\}_{t=1}^T) = \max_k V_T^k$

Traceback:

$$S_T^* = \arg \max_k V_T^k$$

$$S_{t-1}^* = \arg \max_i p(S_t^*|S_{t-1} = i) V_{t-1}^i$$

LEARNING: BAUM-WELCH

- Given HMM with unknown parameters $\theta = \{\{\pi_i\}, \{p_{ij}\}, \{q_i^k\}\}$ and observation sequence $\mathbf{O} = \{O_t\}_{t=1}^T$

find parameters that maximize likelihood of observed data

$$\arg \max_{\theta} p(\{O_t\}_{t=1}^T | \theta)$$

But likelihood doesn't factorize
since observations not i.i.d.

hidden variables – state sequence $\{S_t\}_{t=1}^T$

EM (Baum-Welch) Algorithm:

E-step – Fix parameters, find expected state assignments

M-step – Fix expected state assignments, update parameters

LEARNING: BAUM-WELCH

- Start with random initialization of parameters
- **E-step** – Fix parameters, find expected state assignments

$$\gamma_i(t) = p(S_t = i | O, \theta) = \frac{\alpha_t^i \beta_t^i}{\sum_j \alpha_t^j \beta_t^j}$$

Forward-Backward algorithm

$$\begin{aligned} \xi_{ij}(t) &= p(S_{t-1} = i, S_t = j | O, \theta) \\ &= \frac{p(S_{t-1} = i | O, \theta) p(S_t = j, O_t, \dots, O_T | S_{t-1} = i, \theta)}{p(O_t, \dots, O_T | S_{t-1} = i, \theta)} \\ &= \frac{\gamma_i(t-1) p_{ij} q_j^{O_t} \beta_t^j}{\beta_{t-1}^i} \end{aligned}$$

LEARNING: BAUM-WELCH

- Start with random initialization of parameters
- E-step:

- $\gamma_i(t) = p_{\theta}(S_t = i | O) = \frac{\alpha_t^i \beta_t^i}{\sum_j \alpha_t^j \beta_t^j}$

- $\xi_{ij}(t) = p_{\theta}(S_{t-1} = i, S_t = j | O) = \frac{\gamma_i(t-1) p_{ij} q_j^{O_t} \beta_t^j}{\beta_{t-1}^i}$

- M-step:

- $\pi_i = \frac{\gamma_i(1)}{\sum_{j=1}^K \gamma_j(1)}$

- $p_{ij} = \frac{\sum_{t=2}^T \xi_{ij}(t)}{\sum_{k=1}^K \sum_{t=2}^T \xi_{ik}(t)} = \frac{\sum_{t=2}^T \xi_{ij}(t)}{\sum_{t=2}^T \gamma_i(t)}$

- $q_i^k = \frac{\sum_{t=1}^T \delta_{O_t=k} \gamma_i(t)}{\sum_{t=1}^T \gamma_i(t)}$